



Chapter 4: The Windows App SDK for a UWP Developer

The **Universal Windows Platform (UWP)** was introduced in Windows 10 with the goal of delivering a modern development platform for Windows. Many valuable concepts that are important to provide a modern experience nowadays were included from the beginning, such as support for new input paradigms or advanced UI scaling capabilities.

As such, it shouldn't come as a surprise that the Windows App SDK can be considered a direct successor of UWP, as it reuses many of the same building blocks and fundamental UWP concepts. Some of the Windows App SDK features even come directly from UWP itself. The UI layer is, without a doubt, the most prominent one: WinUI 3 does, in fact, use the same UI layer as UWP but detached from the operating system so that it can also be consumed by Win32 apps developed with .NET or C++.

The result is that if you have experience building apps with UWP, you have a great advantage over a Windows Forms or WPF developer. You will be able to reuse most of your knowledge to build great Windows App SDK applications. However, with UWP being based on a different runtime, when you move to the Win32 ecosystem, there are some important differences around some key concepts, such as the application's life cycle or windows management.

Specifically, we'll cover the following topics in this chapter:

- Moving to a new namespace
- Working with the UI thread
- Controlling the application's life cycle
- Supporting the application's activation
- Application instancing
- Managing the application's window
- Performing operations in the background

By the end of this chapter, you'll have gained the knowledge required, from a UWP perspective, to start new projects or migrate your existing ones.

Technical requirements

The code for the chapter can be found here:

<https://github.com/PacktPublishing/Modernizing-Your-Windows-Applications-with-the-Windows-Apps-SDK-and-WinUI/tree/main/Chapter04>

Moving to a new namespace

When you are building an application with UWP, your home is the **Windows** namespace. Every API that belongs to this ecosystem is included in one of its subnamespaces. One of the most important is **Windows.UI.Xaml**, which includes all the building blocks to create the UI: controls, dependency properties, styles, and so on.

When moving the UI layer to the Windows App SDK, the team had to create a different namespace, to avoid conflicts with the one built inside the operating system. As such, all the UI building blocks for WinUI are included in the **Microsoft.UI.Xaml** namespace. Compared to a traditional UWP application, this introduces the following two main changes:

- If you want to port over some code you have created in UWP, you will have to change every reference to the **Windows.*** namespace to **Microsoft.***.
- When you create a new WinUI application, you will find the following declaration inside the **App.xaml** file:

```
<Application>
  <Application.Resources>
    <ResourceDictionary>
      <ResourceDictionary.MergedDictionaries>
        <XamlControlsResources
          xmlns="using:Microsoft.UI.Xaml
            .Controls" />
      </ResourceDictionary.MergedDictionaries>

      <!--Other app resources here -->
    </ResourceDictionary>
  </Application.Resources>
</Application>
```

This declaration will make sure that every UI control you use in the application (including basic ones, such as **Button** or **TextBlock**) comes from the WinUI library and not from the operating system.

If you want to know more about using namespaces in WinUI, you can refer to [***Chapter 3**, The Windows App SDK for a WPF Developer*](#).

Working with the UI thread

In [***Chapter 3**, The Windows App SDK for a WPF Developer*](#), you already met the **Dispatcher** class. It's a special class provided by the XAML framework that you can use to dispatch a task on the UI thread. Since every WinUI application has a single UI thread, **Dispatcher** becomes especially important when you are executing some code on a background thread but then, at some point, you have to interact with a UI

control. Without using the **Dispatcher** class, the operation would raise an exception, since you can't update the UI from a different thread than the UI one.

UWP exposes a **Dispatcher** class on every page, which belongs to the **Windows.UI.Core.CoreDispatcher** namespace. Here is an example of how to use it:

```
await Dispatcher.RunAsync(Windows.UI.Core.  
    CoreDispatcherPriority.Normal, () =>  
{  
    txtHeader.Text = "This is a header";  
});
```

Actions on the UI thread are dispatched using the **RunAsync()** method, which is asynchronous. The Windows platform, however, offers another API called **DispatcherQueue**, which supports all types of Windows applications: Win32, UWP, and so on. If, in the UWP ecosystem, you had the option to choose between the two, when it comes to the Windows App SDK and WinUI, you must use **DispatcherQueue** since **CoreDispatcher** is built around concepts that are available only in the Windows Runtime ecosystem.

This is an example of how to use **DispatcherQueue**:

```
DispatcherQueue.TryEnqueue(() =>  
{  
    txtHeader.Text = "This is a header";  
});
```

As mentioned earlier, the **DispatcherQueue** class (which belongs to the **Microsoft.UI.Dispatching** namespace) is already available in UWP. However, you must manually obtain a reference to it, since it isn't automatically exposed by XAML controls. In WinUI, instead, this class is directly exposed by the **Window** and **Page** classes, which means that you can use it directly in code-behind. If, instead, you need to use **Dispatcher** in a regular class, you must retrieve a reference first using

the **GetForCurrentThread()** method, as shown in the following example:

```
var dispatcher = DispatcherQueue.GetForCurrentThread();
dispatcher.TryEnqueue(() =>
{
    txtHeader.Text = "This is a header";
});
```

Compared to the **Dispatcher** class in UWP, the **DispatcherQueue** API has a disadvantage: it isn't asynchronous, so you aren't able to wait until the operation is completed before moving on with other tasks. To improve the development experience, you can use the Windows Community Toolkit, as explained in [*Chapter 3, The Windows App SDK for a WPF Developer*](#). Thanks to this library, you will have access to an extension method called **EnqueueAsync()**, which you can use as in the following sample:

```
await DispatcherQueue.EnqueueAsync(() =>
{
    myTextBlock.Text = result;
});
//continue the work
```

Thanks to this asynchronous extension method, you can replicate the behavior of the **RunAsync()** method exposed by the **Dispatcher** class available in UWP.

You can refer to [*Chapter 3, The Windows App SDK for a WPF Developer*](#), for more information on how to integrate the Windows Community Toolkit and this extension.

Now that we know how to properly work with the UI thread, we can move on to another significant difference between UWP and Windows App SDK applications: the application's life cycle.

Controlling the application's life cycle

One of the biggest differences with the UWP model, compared to the more traditional Win32 one, is the application life cycle. Since UWP apps were originally created to run on multiple devices (including less powerful ones compared to a computer, such as mobile phones or tablets), Windows implements a series of optimizations to avoid an application consuming too much memory or CPU, making the system unstable.

Here are some of the key differences compared to the life cycle of a Win32 application:

- **Applications aren't able to run in the background:** A few seconds after an application has been minimized to the taskbar, it gets suspended and isn't able to perform any operations. If you need to perform operations in the background, you must use special features, such as background tasks.
- **When an application is suspended, it doesn't consume CPU (since it can't perform any task) but it still uses memory:** The process, in fact, is kept alive so that if the user goes back to the application, it's instantly resumed and they can continue where they left from. However, if the system is low on memory, Windows can terminate one or more suspended applications so that it can recover enough memory to launch other processes. In this scenario, it's up to the developer to save the state so that when the application is reopened, it can resume the previous status.

These features are powerful because they help to deliver applications that are respectful of the system resources. However, when you are working on enterprise apps, you might need more flexibility, such as being able to run a long-running task in the background. Windows App SDK applications are based on the Win32 model and, as such, they

have a more traditional life cycle that isn't controlled by Windows. For example, they can perform operations in the background without any limitations and they never get terminated, unless an exception happens.

However, this doesn't mean that a Win32 application shouldn't be respectful of the system resources. The main difference compared to UWP is that the responsibility is moved from the operating system to the developer. For this purpose, the Windows App SDK includes a class called **PowerManager**, which belongs to the **Microsoft.Windows.System.Power** namespace and provides information about the current status of the device from a power perspective—whether it's running on battery or AC, how much battery is left, whether the user is present or not, and so on.

As a developer, you can use this information to optimize your application; for example, you can pause heavy tasks if the computer is low on battery, or you can kick out a long-running background process if the user is away. Let's see a brief example of this:

```
private void PerformTask()
{
    if (PowerManager.PowerSourceKind ==
        PowerSourceKind.DC
        && PowerManager.BatteryStatus ==
            BatteryStatus.Discharging &&
            PowerManager.RemainingChargePercent < 25)
    {
        PauseLongRunningWork();
    }
}
```

In the preceding code snippet, you can see three different types of information exposed by the **PowerManager** API:

- **PowerSourceKind**, which tells you whether the computer is running on battery or connected to a power plug
- **BatteryStatus**, which tells you whether the battery is charging or discharging
- **RemainingChargePercent**, which tells you the current charge percentage of the battery

In the previous example, we're using these properties to determine whether it's OK to execute a long-running task—if the computer is running on battery and the charge is below 25%, we pause the execution. Another approach supported by the **PowerManager** API is event subscription. Every property exposed by the API also has an equivalent event, which is triggered when the value changes. For example, the **PowerSource** property is paired with a **PowerSourceKindChanged** event, which is triggered when the computer moves from battery to power adapter and vice versa; or the **RemainingChargePercent** property has a corresponding **RemainingChargePercentChanged**, which is triggered when the battery charge percentage changes.

Let's take a look at the following example for this:

```
public MyPage()
{
    this.InitializeComponent();
    PowerManager.PowerSourceKindChanged +=
        PowerManager_PowerSourceKindChanged;
}
private void PowerManager_PowerSourceKindChanged(object
sender,
    object args)
{
    if (PowerManager.PowerSourceKind ==
PowerSourceKind.DC)
    {
        PauseLongRunningWork();
    }
}
```

```
    }  
    else  
    {  
        ResumeLongRunningWork();  
    }  
}
```

We are applying a logic similar to the one in the previous snippet, except that this time, we're proactively reacting to events. We use the **PowerSourceKindChanged** event to detect when the power source changes and, in the event handler, we use this information to determine whether the computer is running on battery (in this case, we pause the long-running task) or on mains power (in this case, we resume the job).

You can notice how the information about the new status isn't returned as an event argument (**args** is a generic object), but you have to use the **PowerManager** API to retrieve it.

There are other properties that you can retrieve using the **PowerManager** API:

- **DisplayStatus** tells you whether the screen is on, off, or dimmed. You can use this information, for example, to stop rendering UI graphics and start background tasks if the screen is off since it means that the user isn't interacting with the device.
- **EffectivePowerMode** tells you which kind of energy profile is currently applied, such as **Balanced**, **High Performance**, **Battery Saver**, or **Game Mode**.
- **RemainingDischargeTime** returns **TimeSpan** with information on how long the computer can continue to run before the battery is completely discharged.

Thanks to these properties, you can further optimize the execution of tasks that might be more impactful on battery and system performance.

Now it's time to analyze another critical difference between UWP and the Windows App SDK ecosystem: how to manage the way an application gets activated.

Supporting the application's activation

UWP offers a rich set of activation contracts, which enables the application to be launched by multiple actions—file type association, protocol association, sharing, and automatic startup are just a few examples.

To handle these scenarios, the **App** class in a UWP application provides multiple hooks that you can override, such as **OnFileActivated** (when the app is opened using a file type association), **OnShareTargetActivated** (when the app is opened using a share contract), or the more generic **OnActivated**, which can be used to handle all other scenarios for which there isn't a dedicated handler.

The UWP model offers dedicated hooks since, by default, applications are run in a single-instance mode. This means that if the application is running and it gets activated by another action (such as the user double-clicking on a file and then which type is associated with the app), the already activated instance will receive the information about the activation context. The **App** class also provides an event called **OnLaunched**, which, however, is used only to initialize the application at the first launch. It doesn't get invoked every time the application is activated, thus the need to have separate activation events.

In the Windows App SDK application model, instead, applications run as multi-instance by default. Every time the application gets launched, regardless of the activation path, a new instance will be created. This is the default model in the Win32 ecosystem and, on your machine, you can see many examples of applications that behave like this. Take,

for example, the Office suite: every time you double-click on a Word file, it will be opened in a separate instance.

Due to this different model, a Windows App SDK application based on WinUI doesn't expose any more different activation events, but only a single **OnLaunched** event, which will be triggered every time the application gets launched.

Let's take a look at the default **OnLaunched** event of a WinUI application:

```
public partial class App : Application
{
    public App()
    {
        this.InitializeComponent();
    }
    protected override void
    OnLaunched(Microsoft.UI.Xaml
        .LaunchActivatedEventArgs args)
    {
        m_window = new MainWindow();
        m_window.Activate();
    }
    private Window m_window;
}
```

The default implementation is quite simple: it just creates a new instance of **MainWindow** and activates it so that it can be displayed. The Windows App SDK offers a class called **AppInstance**, which belongs to the **Microsoft.Windows.AppLifecycle** namespace, to manage application instances and you can use it to retrieve the activation context.

Let's assume that our application can be associated with one or more file types and, as such, you need to change the **OnLaunched** implemen-

tation to accommodate this requirement. This is an example of how you can do it:

```
protected override void OnLaunched(Microsoft.UI.Xaml.  
    LaunchActivatedEventArgs args)  
{  
    m_window = new MainWindow();  
    var eventArgs =  
        AppInstance.GetCurrent().GetActivatedEventArgs();  
    if (eventArgs.Kind == ExtendedActivationKind.File)  
    {  
        var fileActivationArguments = eventArgs.Data as  
            FileActivatedEventArgs;  
        m_window.FilePath =  
            fileActivationArguments.Files[0].Path;  
    }  
  
    m_window.Activate();  
}
```

As you can notice, we aren't using the **LaunchActivatedEventArgs** parameter of the event, since it doesn't actually contain the information we need to get the context. The right way to retrieve it is by using the **AppInstance** class—after retrieving a reference to the current instance with the **GetCurrent()** method, we can get the activation arguments by calling **GetActivatedEventArgs()**. The first lot of information that we can retrieve from the arguments is the activation type through the **Kind** property (whose type is **ExtendedActivationKind**) since our application might support different activation paths. In the previous example, our application supported only file activation, so we are checking whether the **Kind** property is equal to **ExtendedActivationKind.File**. If that's the case, it means the user has double-clicked on one or more files that we support, so we need to retrieve their path so that we can handle them in the right way. A com-

mon implementation is to redirect users to a dedicated page or window of the application, where they can see the content of the file.

To do this, however, we first have to get the selected files from the **Data** property of the arguments, which contains the activation context. This property is generic across all the various activation paths and, as such, we first have to cast it to the type we're expecting. Since in this case we're working with files, the **Data** property will contain a **FileActivatedEventArgs** object; if, for example, it were a protocol activation, the type would have been **ProtocolActivatedEventArgs**.

Now that we have converted the **Data** property to the correct type, we can use the **Files** collection to retrieve the list of all the files that the user is trying to open. In this example, we're assuming that the user can open only one file at a time, so we retrieve the path of the first item in the collection and store it in a property called **FilePath**, which we have created in the **MainWindow** class.

The window can now use this property to handle the scenario based on the requirements. For example, we can add some logic that uses the file path to load the content of the file and display it on the screen.

We have seen so far how to manage applications that can be activated in different ways, but how can we enable these activation paths? Let's check this in the next section.

Supporting multiple activation paths

If you're using the Windows App SDK packaged model (thus, you're distributing your application using MSIX), you should feel right at home if you have experience with UWP. The way you enable activation paths is the same as for a UWP application—through the manifest. If you double-click on **Package.appxmanifest** in your project and move to the **Capabilities** tab, you will find many activation paths that you can add to your application. For example, if you want to support

files, you can choose **File Type Association** from the drop-down menu and configure it in the following way:

1. Under **Name**, give a meaningful name to the extension, such as the file type you want to support.
2. Under **Supported File Types**, add one or more entries for each type you want to support. The critical information to fill in is the **File type** field, which must contain the extension you want to support (such as **.foo**). This is shown in the following screenshot:

The screenshot shows the 'Declarations' tab in the Windows App SDK configuration tool. The interface includes tabs for 'Application', 'Visual Assets', 'Capabilities', 'Declarations' (selected), 'Content URIs', and 'Packaging'. Below the tabs, a message states: 'Use this page to add declarations and specify their properties.'

Available Declarations: A dropdown menu shows 'Select one...' with an 'Add' button next to it.

Supported Declarations: A list shows 'File Type Associations' with a 'Remove' button next to it.

Description: A text area contains the text: 'Registers file type associations, such as .jpeg, on behalf of the app. Multiple instances of this declaration are allowed in each app.' Below this is a link for 'More information'.

Properties: A section with several input fields: 'Display name:', 'Logo:', 'Info tip:', and 'Name:' (which contains the text 'foo'). Below these is an 'Edit flags' section with two checkboxes: 'Open is safe' and 'Always unsafe'.

Supported file types: A section with a text area containing the text: 'At least one file type must be supported. Enter at least one file type; for example, ".jpg".' Below this is a table with one row: 'Supported file type' (with a 'Remove' button), 'Content type:', and 'File type:' (which contains the text '.foo'). Below the table is an 'Add New' button.

At the bottom, there is a checkbox labeled 'ExecutableOrStartPagelsRequired' which is checked.

Figure 4.1 – The section of the manifest where you can set up custom activation paths, such as file association

When the application is packaged, you can enable all the activation types of UWP, which you can find documented at

<https://docs.microsoft.com/en-us/uwp/api/Windows.ApplicationModel.Activation.ActivationKind>.

Additionally, the packaged model provides a smoother developer experience, since activations are automatically registered when the ap-

plication is deployed and unregistered when the application is removed.

Unpackaged applications, instead, at the time of writing the book, can use four specific activation types supported by the Windows App SDK:

- **Launch**
- **File**
- **Protocol**
- **StartupTask**

Since an unpackaged application behaves like a traditional Win32 application, you can register for these activations directly in the registry, as you would do with a Windows Forms, WPF, or C++ application. For example, this is how you can enable file type association:

<https://docs.microsoft.com/en-us/windows/win32/shell/how-to-register-a-file-type-for-a-new-application>.

However, the Windows App SDK simplifies the registration for these four paths by providing APIs you can invoke when the application is initialized. These APIs will take care, on your behalf, of adding the proper registry keys to enable the activation scenario.

These APIs are exposed by the **ActivationRegistrationManager** class, which belongs to the **Microsoft.Windows.AppLifecycle** namespace. For example, the following snippet shows how to register a file type association:

```
ActivationRegistrationManager.RegisterForFileTypeActivation(new
    string[] { ".foo", ".fee" },
    "logo.png", "File Type Sample",
    new string[] { "View" },
    Process.GetCurrentProcess().MainModule.FileName);
```

The **RegisterForFileTypeActivation()** method requires the following parameters:

- A collection of the file types you want to register. In this sample, we're registering to support the **.foo** and **.fee** extensions.
- Optionally, the path of the image you want to use as an icon for these files. You can also pass an empty string; in this case, the default icon of the app will be used.
- The name of the extension.
- The verbs you want to support.
- The full path of the process will manage these types of extensions. Since in this example the process that will manage the extension is the main one, we're using the **Process** APIs to get the full path of the current process. However, it might also be a different one, for example, if your application is composed of different executables and the activation is managed by a different process than the one that is registering the file type.

If you need to clean up the registration (for example, when the application is about to be uninstalled), you can use the equivalent **unregister** method, as in the following sample:

```
ActivationRegistrationManager.UnregisterForFileTypeActivation  
(new string[] { ".foo", ".fee" },  
Process.GetCurrentProcess().  
MainModule.FileName);
```

The method, called **UnregisterForFileTypeActivation()**, requires the list of extensions you want to unregister and the process path.

This example was based on file type association, but **ActivationRegistrationManager** also provides the following methods:

- **RegisterForProtocolActivation()**: To register one or more protocols (such as `foo://`)
- **RegisterForStartupActivation()**: If you want your application to automatically start together with Windows

All the samples we have seen so far around activation assume that you want to use the default Win32 behavior, which is creating a new instance each time an app is activated. However, there are scenarios where you might want more control over this behavior. Let's learn more in the next section.

Application instancing

As we mentioned in the previous section, UWP apps use, by default, a single-instancing model. If you have an instance of the application running and you try to open a new one using a different activation path (for example, you have double-clicked on an associated file type in File Explorer), the already running instance will be reused. Starting from Windows 10 1803, UWP applications have also gained support for multi-instancing, but this is an opt-in feature.

In the Win32 ecosystem and, as such, in the Windows App SDK as well, the default approach is instead the opposite one: by default, different activation paths will lead to multiple instances of the application being opened.

This behavior is the most flexible one since it enhances the multitasking capabilities of your application. However, there might be scenarios in which you would like to keep using the single-instance model. This is possible thanks to the **AppInstance** class that we introduced in the previous section.

The first step is to create a new class, which will replace the standard initialization of a Windows App SDK application. Right-click on your

project, choose **Add | New class**, and name it **Program.cs**. Now, before working on the class implementation, we must add a special compilation symbol to our project, which will prevent the Windows App SDK from autogenerating the app initialization code and use, instead, the **Program** class we have just created. Right-click on the project, choose **Properties**, move to the **Build** section, and add the following text to the **Conditional compilation symbols** field:

DISABLE_XAML_GENERATED_MAIN

This is how it appears on the screen:

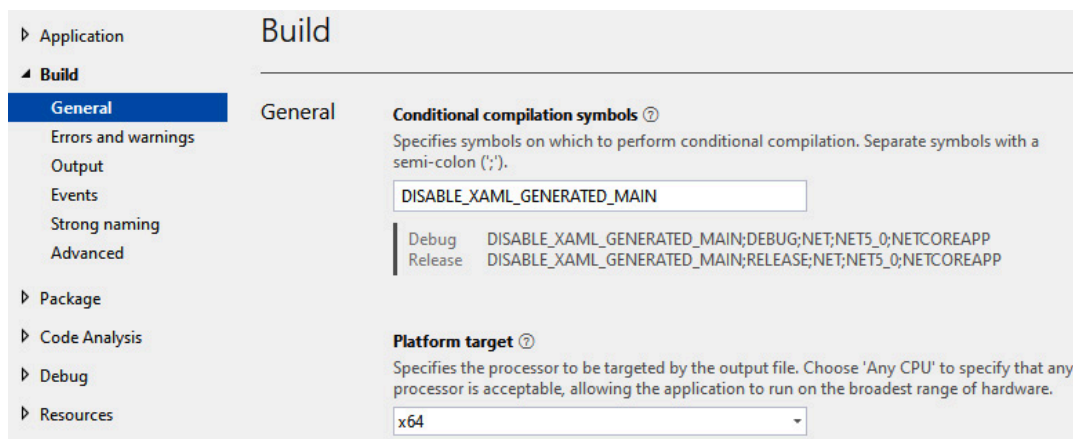


Figure 4.2 – The conditional compilation symbols configuration to enable advanced instancing scenarios

Now we can implement the **Program** class. Let's start with the **Main()** method:

```
public static class Program
{
    [STAThread]
    static async Task<int> Main(string[] args)
    {
        WinRT.ComWrappersSupport.InitializeComWrappers(
    );

        bool isRedirect = false;
        AppActivationArguments activationArgs =
```

```

        AppInstance.GetCurrent().GetActivatedEventArgs();
    }
    AppInstance keyInstance =
        AppInstance.FindOrRegisterForKey("Main");
    if (!keyInstance.IsCurrent)
    {
        isRedirect = true;
        await keyInstance
            .RedirectActivationToAsync(activationArgs);
    }
    if (!isRedirect)
    {
        Microsoft.UI.Xaml.Application.Start((p) =>
        {
            var context = new
                DispatcherQueueSynchronizationContext
                (
                    DispatcherQueue.GetForCurrentThread
                );
            SynchronizationContext
                .SetSynchronizationContext(context);
            new App();
        });
    }
    return 0;
}
}

```

The key to managing the different instances is a method offered by the **AppInstance** class called **FindOrRegisterForKey()**. This method can be used to query Windows to see whether an instance of our application identified by a specific key already exists. In this example, we are labeling the main instance of our application with the **Main** key.

NOTE

*Due to an issue in the Windows App SDK 1.0, the **FindOrRegisterForKey()** method works only if you're targeting the x64 architecture. The issue will be fixed in one of the future Windows App SDK releases, enabling you to also target x86.*

The first time we launch our application, the **IsCurrent** property will be **false** since there won't be any other instance running. In this case, we use the **Microsoft.UI.Xaml.Application.Start()** method to create a new instance starting from the **App** class.

At every other launch of the application (regardless of which is the activation path), the **IsCurrent** property will be **true**, since we already have an instance of the application identified by the **Main** key running in the system. In this case, instead of using the **Start()** method to create a new instance, we call the **RedirectActivationToAsync()** method, which will redirect the initialization to the already existing instance. Notice how, first, we have to retrieve the activation arguments (by calling **AppInstance.GetCurrent().GetActivatedEventArgs()**) so that we can pass them to the **RedirectActivationToAsync()** method. This is important because, if we are in a file activation scenario, for example, the already running instance still needs to know which is the selected file.

If you try now to launch your application multiple times, either via the icon in the Start menu or by activating it via a file or protocol, you won't see multiple instances opening up anymore, but the behavior will be the same as the UWP apps.

However, thanks to the **AppInstance** class, we can enable even more powerful redirection scenarios, as we're going to see in the next section.

Supporting advanced redirection scenarios

As already mentioned in the previous section, Microsoft Word is a good example of an application that supports instancing in a smart way: it always launches a new instance, to maximize its multitasking capabilities, but if you double-click on a file that is already opened, it will redirect you to the already running instance where the file is open.

Thanks to the **AppInstance** class, we can enable similar redirection capabilities in our applications as well. Let's see, for example, how we can change the **Program** class to support this feature:

```
public static class Program
{
    [STAThread]
    static async Task<int> Main(string[] args)
    {
        WinRT.ComWrappersSupport.InitializeComWrappers(
    );

        bool isRedirect = await DecideRedirection();
        if (!isRedirect)
        {
            Microsoft.UI.Xaml.Application.Start((p) =>
            {
                var context = new
                    DispatcherQueueSynchronizationContext
(
                    DispatcherQueue.GetForCurrentThread
    ());

                SynchronizationContext
                    .SetSynchronizationContext(context);
                new App();
            });
        }
    }
}
```

```

    }
    return 0;
}
}

```

The **Main()** method for this code is very similar to the previous one. However, instead of always redirecting any new instance to the existing one, we use a method called **DecideRedirection()** to understand the best approach. Let's take a look at the implementation:

```

private static async Task<bool> DecideRedirection()
{
    bool isRedirect = false;
    // Find out what kind of activation this is.
    AppActivationArguments args =
        AppInstance.GetCurrent().GetActivatedEventArgs();

    ExtendedActivationKind kind = args.Kind;
    if (kind == ExtendedActivationKind.File)
    {
        // This is a file activation: here we'll
        get
        // the file information,
        // and register the file name as our
        instance
        // key.
        if (args.Data is IFileActivatedEventArgs
            fileArgs)
        {
            IStorageItem file = fileArgs.Files[0];
            AppInstance keyInstance = AppInstance
                .FindOrRegisterForKey(file.Name);
            // If we successfully registered the
            file
            // name, we must be the only instance
            // running that was activated for this

```

```
        // file.
        if (keyInstance != null &&
            !keyInstance.IsCurrent)
        {
            isRedirect = true;
            await keyInstance
                .RedirectActivationToAsync(args);
        }
    }
    return isRedirect;
}
```

Inside the method, we check whether we are in a file activation scenario (using the same approach we learned about in the *Supporting the application's activation* section of this chapter). If that's the case, we use the **AppInstance.FindOrRegisterForKey()** method to retrieve a reference to the instance, which might be new or already existing. The key difference is that this time, we aren't using a fixed key (**Main**, in the previous example), but a dynamic one based on the filename. This means that if you are trying to open a file that was already opened, the **IsCurrent** property will be **false**, leading to redirecting the activation to the already running instance. If instead there isn't any instance with the name of the selected file, a new one will be opened.

This is just an example of a redirection scenario. Thanks to the flexibility of the **AppInstance** class, you can enable any other scenario you might need to support in your applications, leading to delivering a better user experience.

Now that we know how to properly manage instances of our application, we are ready to introduce the next topic: working with the application's window.

Managing the application's window

UWP introduced a different model to manage the application's window, compared to the more traditional one offered by the Win32 ecosystem. Since UWP was created with the goal to support multiple devices, the default experience was based on single-window applications. In this context, UWP used the **ApplicationView** and **CoreWindow** classes to host the content of a window. Thanks to these APIs, you were able to perform common tasks related to windows, such as customizing the content and changing the size.

Then, later, when Microsoft started to invest more in the desktop side of the UWP, it introduced support to multiple windows, intending to improve the multitasking story. To better support this scenario, Microsoft introduced a new class called **AppWindow**, which combined the UI thread and the window used by the application to display its content.

In Windows App SDK applications, the windowing experience has some significant differences, since it's based on the **Handle to a Window (HWND)** model, which is the standard approach used by the Win32 ecosystem. HWND is essentially a pointer that you can use to access the low-level window object. To make the developer's work easier, the Windows App SDK provides a class called **AppWindow**, which belongs to the **Microsoft.UI.Windowing** namespace, which abstracts many native features, making it easier to perform operations with the window.

AppWindow has many similarities with the equivalent API in the UWP. As such, if you are using this class in your UWP app, the migration path will be easier for you. If instead you are using **ApplicationView** and **CoreWindow**, there will be more work to do. Regardless of the class

you're using, always keep in mind that the HWND model is very different from the UWP one. As such, even if you're using **AppWindow** in your UWP apps, you won't find a perfect 1:1 mapping across all the APIs and concepts.

Let's see how we can use it.

Using the AppWindow class

Out of the box, the **Window** class in WinUI doesn't give you access to the **AppWindow** abstraction, but you must manually create it starting from the original handle.

You can achieve this goal with the following code snippet:

```
private AppWindow GetAppWindowForCurrentWindow()
{
    IntPtr hWnd =
        WinRT.Interop.WindowNative.GetWindowHandle(this);
    WindowId myWndId =
        Microsoft.UI.Win32Interop.GetWindowIdFromWindow(hWnd);
    return AppWindow.GetFromWindowId(myWndId);
}
```

The first step is retrieving the HWND of the window, using the **GetWindowHandle()** helper method provided and exposed by the **WinRT.Interop.WindowNative** class. The method requires a reference to the window. If you're writing this code in the code-behind of a **Window** class, you can just pass the code itself as a reference (using the **this** keyword); if you are in another class (such as **Page**), you will have to retrieve a reference in another way (for example, by exposing the **Window** property in the **App** class as public).

The next step is to use the HWND to retrieve the identifier of the window using another helper method provided by the

Microsoft.UI.Win32Interop class called **GetWindowIdFromWindow()**.

Once we have the ID, we can finally get the corresponding **AppWindow** class by calling the static **GetFromWindowId()** method exposed by the **AppWindow** class itself.

Now that we have an **AppWindow** object, we can use it to achieve many different tasks.

Moving and resizing the window

Thanks to the **AppWindow** class, you can move the window to a specific position of the screen or resize it. Let's take a look at the following code snippet:

```
private void OnMoveWindow(object sender,
RoutedEventArgs e)
{
    var appWindow = GetAppWindowForCurrentWindow();
    appWindow.Move(new Windows.Graphics.PointInt32(300,
        300));
    appWindow.Resize(new
Windows.Graphics.SizeInt32(800,
        600));
}
```

Once we have used the previous method to retrieve a reference to the **AppWindow** object, we can use the **Move()** method to move the window to a specific location. As a parameter, we must pass a **Windows.Graphics.PointInt32** object, with the *X* and *Y* coordinates of the position.

The **Resize()** method works similarly, except that this time it accepts a **SizeInt32** object with the width and the height we want to set.

The two methods can be used independently. However, if you want to resize and move the window at the same time, you can also use the

convenient **MoveAndResize()** shortcut, as in the following sample:

```
private void OnMoveWindow(object sender,
RoutedEventArgs e)
{
    var appWindow = GetAppWindowForCurrentWindow();
    appWindow.MoveAndResize(new
Windows.Graphics.RectInt32
    (300, 300, 800, 600));
}
```

In this case, the required parameter is **RectInt32** with, as values, the *X* and *Y* values of the coordinates, followed by the width and height.

Customizing the title bar

Another powerful feature supported by **AppWindow** is the ability to customize the title bar of the window. The most basic customization is the title, which can be customized by setting the **Title** property, as in the following example:

```
private void SetTitle()
{
    var appWindow = GetAppWindowForCurrentWindow();
    appWindow.Title = "This is a custom title";
}
```

A more advanced type of customization is the color scheme. The **AppWindow** class offers many properties to customize the colors of the title bar, including the default window buttons (minimize, maximize, and close). Let's take a look at the following code:

```
private void CustomizeTitleBar()
{
    var appWindow = GetAppWindowForCurrentWindow();
    appWindow.Title = "This is a custom title";
    appWindow.TitleBar.ForegroundColor = Colors.White;
```

```
appWindow.TitleBar.BackgroundColor =  
Colors.DarkOrange;  
//Buttons  
appWindow.TitleBar.ButtonBackgroundColor =  
Colors.DarkOrange;  
appWindow.TitleBar.ButtonForegroundColor =  
Colors.White;  
}
```

We are customizing the title bar with an orange background, both for the main bar and the buttons. There are many other properties you can customize, for example, the color of the title bar when the window is inactive or the color of the buttons when they are pressed. The following screenshot shows the result of the previous code:

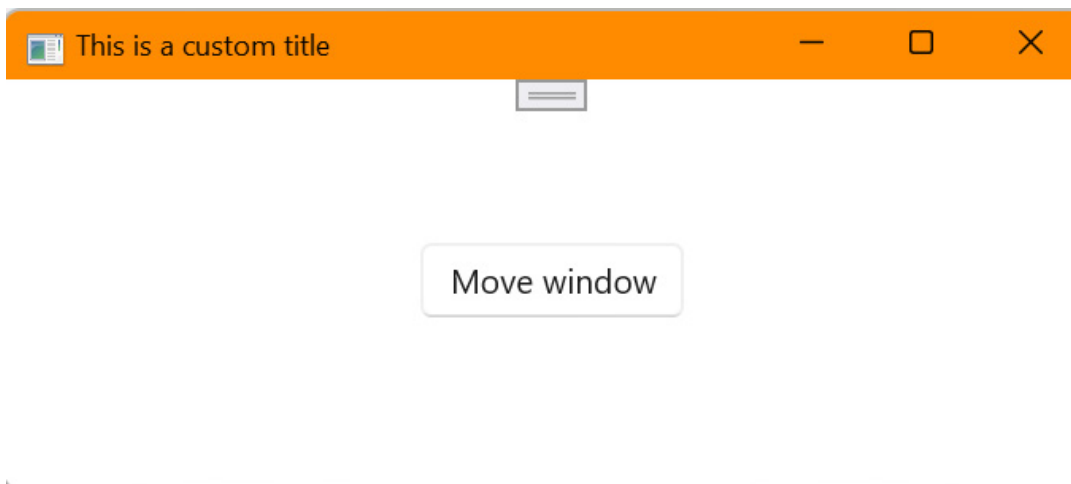


Figure 4.3 – A window with a customized title bar

If you want to have even more flexibility, you can hide the standard title bar and replace it with a XAML control, giving you the freedom of using custom fonts, adding images, and so on. This goal is achieved thanks to a special property of the **AppWindow** class called **ExtendsContentIntoTitleBar**. When it's set to **true**, the content area will use all the available window space, giving you the possibility of defining the title bar as part of your XAML page.

Let's see an example of a XAML page with a custom title bar:

```
<Window>
  <Grid>
    <Grid.RowDefinitions>
      <RowDefinition Height="Auto"/>
      <RowDefinition />
    </Grid.RowDefinitions>
    <Grid Grid.Row="0" x:Name="MyTitleBar"
      Background="LightBlue"
      Visibility="Collapsed">
      <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="Auto" />
      </Grid.ColumnDefinitions>
      <Image x:Name="MyWindowIcon"
        Source="windows.png"
        Grid.Column="0"
        HorizontalAlignment="Left"
        x:FieldModifier="public"
        Width="20" Height="20"
        Margin="12,0"/>
      <TextBlock
        Text="Custom titlebar"
        FontWeight="Bold"
        Grid.Column="0"
        Margin="44,8,0,0"/>
    </Grid>
    <StackPanel Grid.Row="1">
      <TextBlock Text="This is the content" />
    </StackPanel>
  </Grid>
</Window>
```

The window includes a **Grid** control with two rows. The first one contains another **Grid** called **MyTitleBar** that defines the custom title bar,

which is up of an image and text. This control is hidden by default. The second row, instead, contains the actual content of the window.

Now we can use the following code, in code-behind, to replace the default title bar with the **MyTitleBar** control:

```
private void TitlebarCustomBtn_Click(object sender,
    RoutedEventArgs e)
{
    var appWindow = GetAppWindowForCurrentWindow();
    appWindow.TitleBar.ExtendsContentIntoTitleBar =
true;
    // Show the custom titlebar
    MyTitleBar.Visibility = Visibility.Visible;
    //Infer titlebar height
    int titleBarHeight = appWindow.TitleBar.Height;
    this.MyTitleBar.Height = titleBarHeight;
}
```

As the first step, we set **ExtendsContentIntoTitleBar** to **true**, which will enable the content of the window to overflow in the title bar area. Then we make the **MyTitleBar** control visible and, in the end, set its height to be equal to the height of the title bar, to make sure it fits in the available space. Now our title bar will look like this:

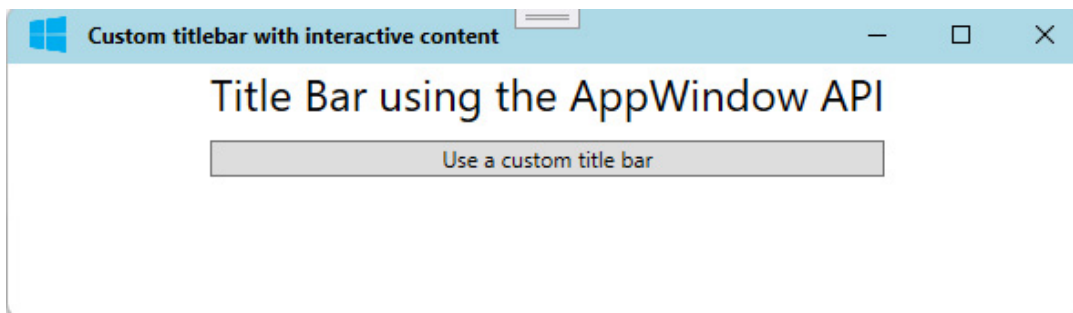


Figure 4.4 – A custom title bar created using a XAML control

In this scenario, you might want to also use properties exposed by the **AppWindow** class to customize the title bar buttons so that they match the look and feel of the custom title bar.

The APIs provided by the **AppWindow** class are very powerful. They enable you to fully customize the title bar but, at the same time, continue to deliver a familiar user experience, by supporting features such as rounded corners in Windows 11 or buttons to minimize, maximize, and close the application.

Using custom presenters

Presenters can be used to customize the way a window is presented to the user. This feature was also supported in UWP but using different APIs and concepts. In the Windows App SDK ecosystem, presenters can easily be enabled thanks to the **AppWindow** class.

At the time of writing, the Windows App SDK supports the following three different presenters:

- **Default:** This is the standard way of presenting a window to the user. The user can fully resize or move the window on the screen, which stays in the foreground as long as the user doesn't move to another application.
- **Fullscreen:** With this configuration, a window is automatically maximized to use the whole screen size. The title bar is also hidden, removing the option for the user to minimize or close the application. The only way to return to the standard user experience is to change back the presenter to the default one. This mode is a good fit for games or kiosk applications.
- **Compact overlay:** With this configuration, the window is reduced to a smaller size at a 16:9 ratio and it can't be resized by the user. However, the window will always be in the foreground. Even if the user moves to another application, the window will stay on top and continue to be visible. This mode is a good fit when you want to enable **Picture-in-Picture (PiP)** scenarios.

To change the presenter, you just have to call the **TrySetPresenter()** method exposed by the **AppWindow** class, passing as a parameter one of

the values supported by the **AppWindowPresenterKind** enumerator.

Let's take a look at an example:

```
private void OnGoFullScreen(object sender,
RoutedEventArgs e)
{
    var appWindow = GetAppWindowForCurrentWindow();
    if (appWindow.Presenter.Kind is not
        AppWindowPresenterKind.FullScreen)
    {
        appWindow.SetPresenter(AppWindowPresenterKind
            .FullScreen);
    }
    else
    {
        appWindow.SetPresenter(AppWindowPresenterKind
            .Default);
    }
}
```

From the preceding code, we see that first, we use the **Presenter.Kind** property exposed by **AppWindow** to check which is the current presenter. If it's the default one (**AppWindowPresenterKind.Default**), then we move to fullscreen by passing **AppWindowPresenterKind.FullScreen** to the **SetPresenter()** method. Otherwise, we return to the default mode.

If you instead want to use the compact overlay mode, you must use the **CompactOverlay** value of the **AppWindowPresenterKind** enumerator, which will enable the following result:

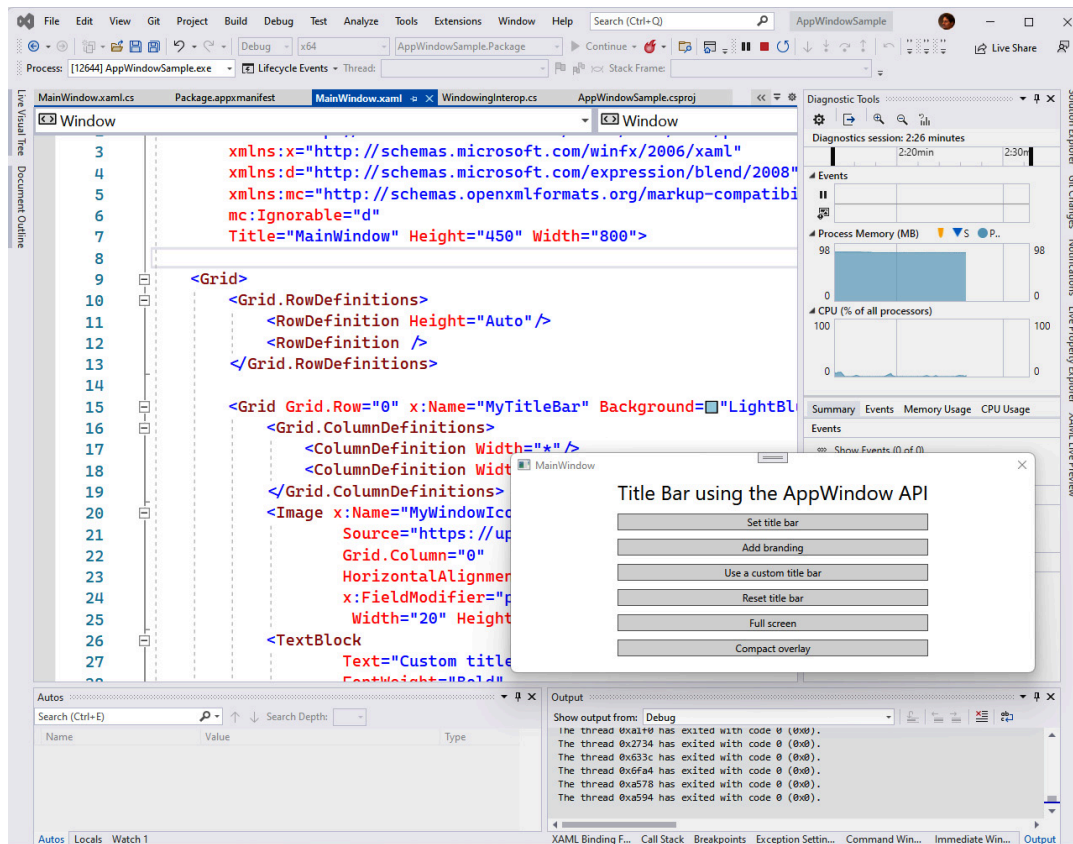


Figure 4.5 – A window using the CompactOverlay presenter

As can be seen in the preceding screenshot, despite Visual Studio being the active application, the app's window is still visible.

We have completed our journey in learning how to take advantage of the **AppWindow** class to manage the window of our application. Now we can move on to analyze another difference between the two ecosystems: background operations.

Performing operations in the background

Since one of the main goals of UWP is to enable developers to write apps that are respectful of the system resources, its background model is quite restrictive compared to the Win32 one. As we highlighted in the *Controlling the application's life cycle* section, applications aren't able to freely run in the background, but they are automatically sus-

pendent after a few seconds of inactivity. Background activities are still supported but through a unique feature called **background tasks**, which are independent snippets of code that can perform operations in the background when a specific trigger happens (every 15 minutes, when the user connects to the internet, and so on).

The Windows App SDK, being part of the Win32 ecosystem, removes most of the restrictions around background activities. The biggest one is that applications aren't suspended anymore when they are minimized, which makes it possible to continue running tasks in the background even when the user isn't actively working with the application.

However, background tasks still have a lot of potential in the Windows App SDK ecosystem, since they make it quite easy to trigger an operation as a consequence of an action or a change in the system. Additionally, background tasks are registered inside the operating system so that they can be executed even if the application that registered them is not running.

For this reason, background tasks are also fully supported by Windows App SDK apps, even if there are a few differences based on the adopted model:

- **Out-of-process:** This is the model that was originally introduced in UWP. The background task is implemented in a separate project (a Windows Runtime component) than the main application and is scheduled by Windows using a dedicated process. This model is supported by Windows App SDK apps in the same way as UWP.
- **In-process:** This model was introduced in Windows 10 1607 to simplify the creation of background tasks. Instead of having to create a different project (which also makes it harder to share code and assets with the main application), the background task is included directly in the main project. An in-process background task is executed by Windows by launching the main process and executing the `OnBackgroundActivated()` method exposed by the **App** class.

Since, as we highlighted in the *Supporting the application's activation* section, now the **App** class only exposes the **OnLaunched()** event, this model is still supported but it must be implemented differently.

With this section, you have now a basic understanding of the differences in managing background operations between UWP and the Windows App SDK. We'll explore the topic in more detail in [*Chapter 8, Integrating Your Application with the Windows Ecosystem*](#), to see how you can enhance your applications with background tasks.

Summary

UWP introduced many concepts that are still truly relevant today—delivering a modern UI experience, optimized for multiple screens and accessibility; being respectful of the system resources; supporting new technologies, such as modern authentication and artificial intelligence. These are just a few examples of features introduced by UWP that are important to build modern desktop applications.

This is the reason the Windows App SDK and WinUI were built around the same principles, but on top of the Win32 ecosystem to give developers more flexibility and make it easier for them to reuse the existing code and libraries.

Even if many of the building blocks are the same, moving to the Win32 ecosystem means that there are some differences around some key concepts, such as the application's life cycle or windowing management. In this chapter, we learned about these differences in detail as well as what the new APIs you must use to properly support these scenarios in Windows App SDK apps are.

This chapter concludes the overview of the Windows App SDK ecosystem for the different development frameworks: Windows Forms, WPF,

and UWP.

In the next chapter, we'll learn how to properly design a Windows App SDK application, by adopting the right navigation pattern or implementing responsive layouts.

Questions

1. To ensure stability and performance, Windows can automatically terminate the Windows App SDK if it is consuming too many resources. True or false?
2. There is no difference between a packaged and an unpackaged application when it comes to supporting activation contracts. True or false?
3. Windows App SDK applications, unlike UWP apps, support multi-instancing by default. True or false?